

DSM Bonus Mini Project

Kindly read the following rules and regulations carefully before asking questions and clarifications.

- As mentioned, this project is meant to be a bonus project. I.e, not doing the project will not cost you at all.
- It is an individual project and not related to your groups/ labs.
- Successful completion of the project will fetch a maximum of 5% of the total grade, which is to be added to the overall aggregate obtained in the course by combining all other components. The total thus obtained shall be capped at 100%.
- This project will require the knowledge of all concepts learnt so far in the course. It will be an amazing learning opportunity and a nice way to challenge yourself. Building on top of this project can also get you your first resume worthy project, especially for people looking to go into digital hardware design.
- **In case you opt for the project and fail to make substantial progress, you shall be penalized for 2% of your course total. This is non-negotiable and no exceptions shall be dealt with.** It will only be applied if the project shows negligible work and total reliance on LLM generated content. **An attempt which shows the notable effort put in won't receive the penalty even if the circuit doesn't work in the end.**
- The final evals for the project will tentatively be from the **1st to 4th December**. It is mandatory to be on campus for the eval. The slots can be adjusted, but no online slot shall be provided.
- Simply completing the project does not guarantee the entire 5%. There will be several grading factors. **However, designing a working circuit does guarantee you a minimum of 2%.**
- The evals will be taken by TAs. Anyone who feels that the TAs might not be "competent enough" or that they won't be able to appreciate your brilliance, can choose not to attempt this project.
- Considering the timeline of the project, it is possible that TAs will not be able to help you with the project actively.

Project: Basic Integer Calculator Circuit Design

Inputs:

- 2 bit select signal to choose one amongst: 00 = ADDITION ($A+B$), 01 = SUBTRACTION ($A-B$), 10 = MULTIPLICATION ($A*B$), 11 = DIVISION (A/B)
- Two unsigned integers A, B ($0 \leq A, B \leq 255$), each given as 8-bit binary inputs
- Clock input (with a push button)
- RESET signal: changes the output to 0 if high asynchronously. Before any operation, we toggle this switch.

Note: A , B and the select signal must be held steady until the Done signal goes high. Changing them mid-operation aborts the current operation (Done goes low) and restarts it.

Outputs:

- Result / Quotient: the main output, whose width depends on the selected operation. ADDITION gives the sum, SUBTRACTION gives the magnitude $|A-B|$, MULTIPLICATION gives the product and DIVISION gives the quotient Q for A/B .
- Remainder for A/B (valid only for DIVISION; 0 for the other operations)
- Negative flag (LED): Turns high only for SUBTRACTION when $A < B$. The display shows the magnitude, e.g. $A=3$, $B=10$ shows 7 on the display with the negative LED on, i.e. -7.
- Done signal: turns high when the output is fully generated (for DIVISION, both Q and R)
- Error signal: turns high when the input has an error (e.g. division by zero, $B = 0$)
- 1 Counter Display Module which shows the number of clocks needed to get the result after some input is given
- Decimal 7-segment displays for the final answer: one set for the Result / Quotient and a separate set for the Remainder, both shown at the same time. This requires a binary-to-BCD converter.

Assume any number of bits that gives a correct answer for quotient and remainder. Having segment displays and leds for signals at the output is advisable.

Performance Criteria (Based on priority):

1. Worst case number of clock cycles needed to generate the output, over all four operations and all valid inputs, counted from the moment an input changes until Done goes high (The maximum number of clock cycles needed decides the worst case delay for the circuit).
2. Hardware Simplicity (total NAND gates of the entire circuit, including the BCD converter, displays and LEDs)

Note: the multiplication and division methods are left to you; you are expected to justify your choice.

Instructions for the project:

- The circuit needs to be made and simulated on sebastian.itch.io/digital-logic-sim . It contains a nand block that you can use. Any other required block needs to be implemented using nand blocks. Keep a count of your total nand gates! **More resources regarding this will be shared in some time.**
- You can use any module of choice, but each module needs to be made using basic blocks. The basic blocks that can be used to make a sub circuit are as follows: logic gates (NOT, AND, OR, NAND, NOR, XOR, XNOR), Mux, Demux, Encoders, Decoders, HA, FA, Latches, FFs (D, T, JK), Registers, Segment display. **Any module not mentioned above, that is used and not defined, can be penalized.**
- A clean circuit with a neat input output design is highly recommended. It will be significantly easier to debug and explain during the final presentation. Labels and tunnels should be used wherever needed.
- Use temporary LEDs for intermediate debugging of the circuit.
- Try taking a top down or a bottom to up approach, whatever seems easier. This means start at a high level and keep replacing it with small blocks that achieve the same functionality, till you reach the basic blocks.
- For the final presentation, you are required to make a ppt (can be llm generated but be cautious about the contents) and present. There is no minimum or maximum time limit. A rough estimate can be 15-20 minutes of presentation plus an equal amount of cross questioning at every point. (You can be grilled on any single word that you write so make sure to proofread whatever you generate using AI)

Things to add in the ppt (Essentials):

1. Algorithms used for all four operations (ADD, SUB, MUL, DIV) and for the binary-to-BCD conversion
2. All sub circuits defined and used within the circuit
3. Block diagram of the circuit
4. Performance:
 - a. Worst case number of clock cycles, for each operation and overall.
 - b. Number of NAND gates to make the entire circuit, with relevant calculations. You need not write the calculations, just an intuitive idea is enough**Note that these will be checked and incorrectly reporting parameters shall be penalized so be careful with this.**
5. Could you improve on your circuit in any way that reduces clock cycles or hardware usage and how?
6. Sources used to make the project (links to all, including the LLM chat history)
7. What did you learn from the project and if it was worth the effort put in.
8. Was this mini project a good idea? How can this be improved in the upcoming years?

The ppt should not be cluttered with information. It is your job to make a minimal ppt that can support you during the presentation.

Submissions:

1. Fill the form to opt for the project (Deadline: 15th October)
2. A publicly accessible simulation (Tentative deadline: 16th November 11:59 p.m.)
3. Ppt for the presentation (Tentative deadline: 2nd December 11:59 a.m.)

Doubts:

In case of any doubts related to the above, please put the query on this notion link. PLEASE DON'T DM RANDOM TAs. Putting the query here is much more reliable

Link:

https://app.notion.com/p/Bonus-Project-Queries-3efc6d9a74138049b43effd0e609626d?source=copy_link

To opt for the project, fill this form you must fill the following form. MAKE SURE TO CONSIDER THE PRO AND CONS OF THE PROJECT AND NOT FOLLOW THE HERD.

DEADLINE: 15th October

https://docs.google.com/forms/d/e/1FAIpQLSeivV2_azpAsiAED9GqLuFNac3KSuigxjP6q0wkLfNi3WS10w/viewform?usp=publish-editor

Further Prospects (Ungraded, but advisable to add for resume if interested):

1. *Try to add another function that helps temporarily store the result of an operation, so that you can work on that obtained result again. Eg: $(a+b)/c$*
2. *Change the output to entirely 2's complement so that any arithmetic can be performed. This will require a larger result data width, since multiplication and division cover a huge range of outputs, that too including negative numbers including negative numbers,*
3. *Try introducing a RAM Module in the ckt, i.e introduce two more functions load and store. This will help you store multiple values in your calculator and access different values depending on the address that can be given as another input. (You almost have a processor at this point)*
4. *Finally add a ROM module which is designed based on the select lines, and you will have made a totally unique processor!!!*
5. *Expand the functions (or more accurately, the instructions) that your processor can execute (maybe add logical instructions and comparators)*
6. *Add an instruction counter, which would help introduce branching and looping instructions (if-else, for, while).*
7. *Read about pipelining.*